

JavaScript, структуры данных

Строки

В JavaScript любые текстовые данные являются **строками**.

Внутренним форматом строк, вне зависимости от кодировки страницы, является Юникод (Unicode).

Строки заключаются в двойные или одинарные кавычки (в JavaScript между ними нет разницы), или обратные кавычки.

В JavaScript строки рассматриваются как константы, то есть нельзя изменить часть строки.

Соединение строк

Соединение строк осуществляется с помощью оператора "+".

```
let str1 = 'Script';  
let str2 = 'Java';  
  
alert(str2 + str1); // выведет JavaScript
```

Длина строки

Длина строки определяется с помощью метода **length**.

```
// строка из 7 символов, последний - перевод строки  
let str = 'Script\n';  
  
alert(str.length ); // выведет 7  
  
alert('JavaScript'.length ); // выведет 10
```

Обращение к элементу строки

Осуществляется двумя способами:

- с помощью метода **charAt()**;
- с помощью квадратных скобок (**[]**).

В скобках указывается номер нужного символа, нумерация символов начинается с 0.

Например:

```
let str = 'Script';
```

```
alert( str.charAt(0) ); // выведет S
```

```
alert( str[3] ); // выведет i
```

Обращение к элементу строки

Разница между **charAt()** и квадратными скобками в том, что если осуществляется обращение к несуществующему символу:

- **charAt()** выдает пустую строку;
- **[]** – неопределенное значение **undefined**.

Например:

```
let str = 'JavaScript';
```

```
alert( ''.charAt(0) ); // пустая строка
```

```
alert( ''[0] ); // undefined
```

```
alert( str.charAt(20) ); // пустая строка
```

```
alert( str[20] ); // undefined
```

Строка – это константа

В JavaScript строки рассматриваются как константы, то есть нельзя присвоить новое значение одному элементу строки:

```
let str = 'Script';  
str[1]='r';  
alert(str); //выведет исходную строку Script
```

Сравнение строк

Сравнение строк осуществляется в **лексикографическом порядке** или посимвольно.

Сравнение символов осуществляется по их Unicode коду. Для того чтобы получить **код символа** используется метод:

`charCodeAt(позиция)`

который возвращает код символа в Unicode на указанной позиции.

Например:

```
alert( 'JavaScript'.charCodeAt(1) ); // код английской а - 97  
alert( 'Программа'.charCodeAt(5) ); // код русской а - 1072  
alert( 'JAVASCRIPT'.charCodeAt(1) ); // код английской А - 65  
alert( 'ПРОГРАММА'.charCodeAt(5) ); // код русской А - 1040
```


Сравнение строк

Сравнение строк **str1** и **str2**, например **str1 > str2**, в лексикографическом порядке осуществляется следующим образом:

1. Сравниваются первые символы **str1[0]** и **str2[0]**. Если они разные, то сравниваются они и, в зависимости от результата, возвращается **true** или **false**. Если же они одинаковые, то переходим к пункту 2.
2. Сравниваются вторые символы **str1[1]** и **str2[1]** аналогично первым символам, если они одинаковые – то пункт 3.
3. Сравниваются третьи символы **str1[2]** и **str2[2]** и так далее, пока символы не станут, наконец, разными, и тогда какой символ больше – та строка и больше. Если же в какой-либо строке закончились символы, то считается, что она меньше, а если закончились в обеих – они равны.

Сравнение строк

Например:

`"JavaScript" > "Java"` - `true`, т.к. начало совпадает, но в 1-й строке больше символов

`"JavaSCRIPT" < "JavaScript"` - `true`, т.к. начало совпадает, но `'C' < 'c'`

Числа в виде строк сравниваются как строки, поэтому сравнение может выдать неожиданные результаты:

`"2" > "100"` - `true`, т.к. `"2" > "1"`

Поиск подстроки

Для поиска подстроки используются методы:

`indexOf(подстрока[, начальная_позиция])`

возвращает позицию, начиная с которой найдена подстрока, или -1, если ничего не найдено. Второй параметр является необязательным, его наличие позволяет осуществлять поиск подстроки с указанной начальной позиции.

`lastIndexOf(подстрока[, начальная_позиция])`

ищет подстроку не с начала, а с конца строки.

Поиск подстроки

Например:

```
let str = 'Я изучаю языки Java и JavaScript';

alert(str.indexOf('Java')); // 15, номер первого вхождения слова
alert(str.indexOf('java')); // -1, так как учитывается регистр
alert(str.indexOf('я')); // 9, так как первое Я написано с большой буквы

alert(str.indexOf('Java', 16)); // 22, номер второго вхождения слова

alert(str.lastIndexOf('Java')); // 22, номер второго вхождения слова

alert(str.lastIndexOf('Java', 16)); // 15, номер первого вхождения слова
```

Смена регистра

Методы **toLowerCase()** и **toUpperCase()** преобразуют все буквы к маленьким (строчным) и большим (прописными) соответственно.

Например:

```
str = 'JAVASCRIPT';  
alert(str.toLowerCase() ); // javascript
```

Выделение подстроки

В JavaScript существуют три метода для выделения подстроки:

substring(начало [, конец])

Метод возвращает подстроку с позиции начало до, но не включая, конец. Если параметр конец отсутствует, то возвращается строка, начиная с позиции начало и до конца строки.

```
let str = 'JavaScript';  
  
// символы с позиции 4 по 7, не включая 7.  
alert(str.substring(4,7)); // Scr  
  
// символы с позиции 4 до конца  
alert( str.substring(4) ); // Script,
```

Выделение подстроки

В JavaScript существуют три метода для выделения подстроки:

`substr(начало [, длина])`

В этом методе первый аргумент имеет такой же смысл, как и в **substring()**, а второй содержит не конечную позицию, а количество выделяемых символов. Если второй параметр отсутствует – подразумевается выделение «до конца строки».

```
let str = JavaScript;
```

```
alert(str.substr(4,3)); // Scr, символы с позиции 4 длиной 3
```

```
alert(str.substr(4)); // Script, символы с позиции 4 до конца
```

Выделение подстроки

В JavaScript существуют три метода для выделения подстроки:

`slice(начало [, конец])`

Метод аналогичен **substring()**, возвращает часть строки от позиции начало до, но не включая, позиции конец.

```
let str = 'JavaScript';
```

```
// символы с позиции 4 по 7, не включая 7.
```

```
alert(str.slice(4,7)); // Scr
```

```
// символы с позиции 4 до конца
```

```
alert( str.slice(4) ); // Script,
```


Выделение подстроки

В **substring()** отрицательные аргументы интерпретируются как равные нулю. Слишком большие значения усекаются до длины строки. Кроме того, если начало больше, чем конец, то аргументы меняются местами:

```
let str = 'JavaScript';

alert(str.substring(-4)); // выведет JavaScript, -4 заменяется на 0

alert(str.substring(4, -2)); // Java, -2 заменяется на 0, 0 и 4
меняется местами

alert(str.substring(4, 20)); // Script, 20 заменяется на 10 (длина строки)
```

Выделение подстроки

В методе **slice()** отрицательные значения отсчитываются от конца строки:

```
let str = 'JavaScript';

alert(str.slice(-6)); // Script, 6 отсчитывается от конца строки

alert(str.slice(0, -6)); // Java, от начала строки до 6 символа от конца строки
```

Удаление пробельных символов

Пробельными символами считаются: пробел, табуляция, неразрывный пробел и прочие.

Для удаления пробельных символов используются следующие методы:

- **trim()** удаляет пробельные символы с начала и конца строки;
- **trimRight()** удаляет пробельные символы с конца строки;
- **trimLeft()** удаляет пробельные символы с начала строки.

Например:

```
let text = '    Я изучаю JavaScript    ';  
  
alert( text.trim() ); // результат 'Я изучаю JavaScript'  
  
alert( text.trimRight() ); // результат '    Я изучаю JavaScript'  
  
alert( text.trimLeft() ); // результат 'Я изучаю JavaScript    '
```

Пример

Пример. Создадим функцию, которая разбивает строку на отдельные слова и формирует новую строку с обратным порядком слов.

```
function reverseString(str) {  
    let pos = 0;  
    let oldPos = 0;  
    let newStr = '';  
  
    str = str + ' ';  
  
    while (str.indexOf(' ', pos + 1) !== -1) {  
        oldPos = pos; //начало очередного слова  
        pos = str.indexOf(' ', pos) + 1; //конец очередного слова  
        newStr = str.slice(oldPos, pos) + newStr;  
    }  
    return newStr;  
}
```

Строки

Задание 1. Создать функцию, которая в качестве параметра получает строку, состоящую из слов и чисел, а возвращает сумму чисел этой строки.

Массивы

Массив с числовыми индексами – это многомерное упорядоченное множество объектов, обращение к которым ведется при помощи целочисленных индексов.

Они обычно используются для хранения упорядоченных коллекций данных, например – списка товаров на странице, студентов в группе и т.п.

Одномерный массив

Одномерный массив – это упорядоченное множество объектов, обращение к которым ведется при помощи целочисленного индекса.

Для создания массива используются квадратные скобки. Элементы массива в квадратных скобках перечисляются через запятую.

```
let arrBuy = []; // создание пустого массива
```

```
let arrGoods = ['карандаш', 'ручка', 'тетрадь']; // создание массива с указанием начальных значений
```

Допускается, но не рекомендуется, создавать массив, в котором элементы относятся к разному типу:

```
let arrTest = [3.14, true, 85, 'word'];
```

Одномерный массив

```
let arrGoods = ['карандаш', 'ручка', 'тетрадь'];
```

Обращение к массиву осуществляется **по числовому индексу**, нумерация элементов начинается с нуля.

```
alert(arrGoods[1]); // ручка
```

Допускается **изменение значения массива по его индексу** с помощью оператора присваивания.

```
arrGoods[0] = 'альбом'; // массив состоит из элементов 'альбом', 'ручка', 'тетрадь'
```

Добавление нового элемента

```
let arrGoods = ['карандаш', 'ручка', 'тетрадь'];
```

Занести новое значение можно на любое место массива, при этом пропущенные элементы считаются равными **undefined**.

```
arrGoods[3] = "карандаш";
```

```
arrGoods[5] = "блокнот";
```

```
alert(arrGoods); //результат -  
"карандаш,ручка,тетрадь,карандаш,,блокнот"
```

```
alert(arrGoods[4]); //результат - undefined
```

Длина массива

Для определения длины массива используется метод **length**, который выдает номер последнего заполненного элемента плюс один:

```
let arrGoods = ['карандаш', 'ручка', 'тетрадь'];
```

```
alert(arrGoods.length); //результат - 3
```

Занесем в массив несколько элементов с пропуском:

```
let arrGoods = ['карандаш', 'ручка', , 'тетрадь'];
```

```
alert(arrGoods.length); //результат - 4
```

Длина массива - 4, несмотря на то, что всего заполненных элементов в массиве - 3, а один элемент пропущен.

Перебор элементов массива

Пример. Найдем сумму элементов массива.

```
let arr = [12, -2, 6, 8, 12, 7, 1];

let sum = 0;
for(let i in arr) {
    sum += arr[i];
}

alert(sum) //результат - 44
```

Перебор элементов массива

Также для перебора элементов массива используется метод **forEach()**.

Метод **forEach()** в качестве параметра получает функцию, которая выполняется для каждого элемента массива.

В эту функцию можно передавать 3 параметра, если эти параметры есть (они необязательны):

- первый параметр - элемент массива;
- второй - его номер в массиве (индекс);
- третий - сам массив.

forEach()

Синтаксис метода:

```
массив.forEach( function(элемент) {  
    операторы, которые выполнятся  
    для всех элементов массива  
})
```

forEach()

Пример. Создадим список на основе элементов массива.

```
let arrLang = ['Python', 'JavaScript', 'Java', 'C++', 'C#'];  
  
document.write('<ul>');  
  
arrLang.forEach( function(elem) {  
    document.write('<li>${ elem }</li>');  
});  
  
document.write('</ul>');
```

forEach()

Синтаксис метода:

```
массив.forEach( function(элемент, индекс) {  
    операторы, которые выполнятся  
    для всех элементов массива  
})
```

forEach()

Пример. Создадим список из гиперссылок на основе элементов массива. Имя файла – это строка **lang + номер_элемента + .html**.

```
let arrLang = ['Python', 'JavaScript', 'Java', 'C++', 'C#'];  
  
document.write('<ul>');  
  
arrLang.forEach( function(elem, index) {  
    document.write(`<li><a href="lang${index}.html">  
        ${ elem }  
    </li>`);  
});  
document.write('</ul>');
```

Метод `split()`

Для разбиения строки по указанному разделителю и формирования массива из выделенных частей, используется метод **`split()`**.

Его синтаксис:

- **`split()`** - возвращает массив содержащий один элемент, состоящий из всей строки;
- **`split(разделитель)`** - разбивает строку по указанному разделителю и заносит полученные подстроки в массив;
- **`split(разделитель, количество)`** - разбивает строку по указанному разделителю и заносит в массив указанное количество подстрок.

Метод `split(разделитель)`

`split(разделитель)` - разбивает строку по указанному разделителю и заносит полученные подстроки в массив.

Особенности использования разделителя:

- в качестве разделителя может использоваться один символ:

```
let str = 'Я изучаю JavaScript';
```

```
// разбиваем строку на слова по пробелам
```

```
let arr = str.split(' ');
```

```
// результат ['Я', 'изучаю', 'JavaScript']
```

Метод split(разделитель)

- в качестве разделителя может выступать строка:

```
let str = '<ul><li>Я</li><li>изучаю</li><li>JavaScript</li> ></ul>';

let arr = str.split('<ul><li>');
// результат ['', 'Я</li><li>изучаю</li><li>JavaScript</li></ul>']

str = arr[1]; // 'Я</li><li>изучаю</li><li>JavaScript</li></ul>'

str = str.split('</li></ul>')[0];
// результат 'Я</li><li>изучаю</li><li>JavaScript'

arr = str.split('</li><li>');
// результат ['Я', 'изучаю', 'JavaScript']
```

Метод split(разделитель)

Особенности использования разделителя:

- в качестве разделителя можно использовать пустую строку, тогда строка будет разбита на символы:

```
let str = 'JavaScript';

let arr = str.split('');
// результат ['J', 'a', 'v', 'a', 's', 'c', 'r', 'i', 'p', 't']
```


Метод split(разделитель)

Если задан второй параметр метода:

split(разделитель, кол-во)

в массив добавляется указанное количество выделенных подстрок.

```
let str = 'JavaScript';
```

```
let arr = str.split(' ', 4); // результат ['J', 'a', 'v', 'a']
```

Метод split

Задание 2. Дан фрагмент HTML -кода, вывести содержимое всех абзацев кода.

Например, рассматривается следующий HTML-код:

```
<ul>
  <li>элемент 1</li>
  <li>элемент 2</li>
</ul>
<p>Первый абзац</p>
<br>
<p>Второй абзац</p>
<div>
  <p>Третий абзац</p>
  <p>Четвертый абзац</p>
</div>
```

Метод join()

Для преобразования массива в строку используется метод **join()**.

Синтаксис:

- **join()** - возвращает строку, состоящую из элементов массива, разделенных запятой

```
let arr = ['Я', 'изучаю', 'JavaScript'];
```

```
let str = arr.join() // результат 'Я,изучаю,JavaScript'
```

- **join(разделитель)** - возвращает строку, состоящую из элементов массива, разделенных указанным разделителем:

```
let arr = ['Я', 'изучаю', 'JavaScript'];
```

```
let str = arr.join(' ') // результат 'Я изучаю JavaScript'
```

Пример

Пример. Создадим функцию, которая убирает из строки "лишние" пробелы.

Реализуем алгоритм замены в строке двух пробелов на один, до тех пор, пока в строке есть два пробела.

```
function delDoubleSpaces(str) {  
    while(str.indexOf('  ') >= 0) {  
        let arr = str.split('  ');  
        str = arr.join(' ');  
    }  
    return str;  
}
```

Добавление элементов

Для добавления нового элемента в массив **используются методы**:

- **push(элемент)** - добавляет элемент в конец массива:

```
let arrGoods = ['карандаш', 'ручка', 'тетрадь'];  
  
arrGoods.push('альбом');  
// результат - ['карандаш', 'ручка', 'тетрадь', 'альбом'];
```

Добавление элементов

Для добавления нового элемента в массив **используются методы**:

- **unshift(элемент)** - добавляет элемент в начало массива:

```
let arrGoods = ['карандаш', 'ручка', 'тетрадь'];  
  
arrGoods.unshift('альбом');  
// результат - ['альбом', 'карандаш', 'ручка', 'тетрадь'];
```

Добавление элементов

Методы **push()** и **unshift()** позволяют добавлять в конец и начало массива соответственно сразу несколько элементов:

```
let arrGoods = ['карандаш', 'ручка', 'тетрадь'];

arrGoods.push('альбом', 'блокнот');
// результат - ['карандаш', 'ручка', 'тетрадь', 'альбом', 'блокнот']

arrGoods.unshift('ластик', 'корректор');
/* результат ['ластик', 'корректор', 'карандаш', 'ручка', 'тетрадь',
               'альбом', 'блокнот']
*/
```

Удаление элементов

Для удаления элемента из массива **используются методы:**

- **pop()** - удаление последнего элемента:

```
let arrGoods = ['карандаш', 'ручка', 'тетрадь'];

arrGoods.pop(); // ['карандаш', 'ручка']
```

Удаление элементов

Для удаления элемента из массива **используются методы:**

- **shift()** - удаление первого элемента массива:

```
let arrGoods = ['карандаш', 'ручка', 'тетрадь'];  
arrGoods.shift(); // результат - ['ручка', 'тетрадь']
```

Удаление элементов

Удаление последнего элемента массива можно реализовать через **установку новой длины** этого массива.

```
let arrGoods = ['карандаш', 'ручка', 'тетрадь'];  
arrGoods.length --;  
alert(arrGoods); // результат - карандаш, ручка
```

Удаление элементов

«Укоротить» массив на несколько элементов также можно установив новую длину.

```
let arrGoods = ['карандаш', 'ручка', 'тетрадь'];  
arrGoods.length -= 2;  
alert(arrGoods); // 'карандаш'
```

Удаление и вставка

Метод **splice()** – это универсальный метод работы с массивами. С его помощью можно: удалять элементы, вставлять элементы, заменять элементы, по очереди и одновременно.

Его синтаксис:

```
splice(позиция[, кол_элементов, новыйЭлем1,... , новыйЭлемN])
```

С помощью этого метода можно удалить из массива заданное количество элементов, начиная с указанной позиции, а затем вставить новые элементы на их место.

Практически все параметры, кроме первого, могут отсутствовать.

При этом метод возвращает массив из удаленных элементов.

Удаление и вставка

Например, удалим из массива элемент с номером 1.

```
let arrGoods = ['карандаш', 'ручка', 'тетрадь', 'блокнот', 'альбом'];  
arrGoods.splice(1, 1);  
alert(arrGoods); // выведет: карандаш, тетрадь, блокнот, альбом
```

Удаление и вставка

Поскольку метод возвращает удаленные элементы в виде массива, можно удаленный элемент вставить в другое место массива.

Для вставки без удаления вместо параметра, обозначающего количество удаляемых элементов, нужно использовать 0.

```
let arrGoods = ['карандаш', 'ручка', 'тетрадь', 'блокнот', 'альбом'];  
  
let arrDelete = arrGoods.splice(1, 1);  
// arrGoods: карандаш, тетрадь, блокнот, альбом  
  
arrGoods.splice(3, 0, arrDelete[0]);  
// arrGoods: карандаш, тетрадь, блокнот, альбом, ручка
```

Удаление и вставка

Допускается использование отрицательного номера позиции, которая в этом случае отсчитывается с конца:

```
let arrGoods=[ 'карандаш', 'ручка', 'тетрадь', 'блокнот', 'альбом' ];  
  
let arrDelete = arrGoods.splice(-3,2);  
  
alert(arrGoods); // выведет: карандаш, ручка, альбом  
  
alert(arrDelete); // выведет: тетрадь, блокнот
```

Поиск в массиве

Метод **includes()** возвращает логическое значение **true** или **false** и позволяет определить, есть ли в массиве указанный элемент. Также можно указать, с какого индекса начинать поиск:

```
arr.includes(значение, [индекс]);
```

Метод **includes()** использует строгое сравнение (то есть с учетом типов).

Например, при сравнении 2 и '2' метод выдаст **false**.

Поиск в массиве

Пример

```
let arr = [12, 24, 13, 24, 18, 9];  
alert(arr.includes(24));    // true  
alert(arr.includes('24'));  // false  
alert(arr.includes(10));    //false  
alert(arr.includes(24, 4)); // false  
alert(arr.includes(24, 2)); // true
```

Метод `indexOf()`, `lastIndexOf()`

Метод **`indexOf()`** возвращает индекс первого вхождения элемента, если он в массиве найден, и -1, если элемента в массиве нет. Также можно указать, с какого индекса массива начать поиск:

```
arr.indexOf(значение, [индекс]);
```

Метод **`lastIndexOf()`** возвращает индекс последнего вхождения элемента, если он в массиве найден, и -1, если элемента в массиве нет. Также можно указать, с какого индекса массива начать поиск:

```
arr.lastIndexOf(значение, [индекс]);
```

Методы **`indexOf()`** и **`lastIndexOf()`** использует строгое сравнение.

Метод indexOf(), lastIndexOf()

Пример

```
let arr = [12, 24, 13, 24, 18, 9];

alert(arr.indexOf(24));      // результат: 1
alert(arr.indexOf('24'));   //результат: -1
alert(arr.lastIndexOf(24)); //результат: 3
alert(arr.indexOf(24, 4));  // результат: -1
alert(arr.lastIndexOf(24, 4)); // результат: 3
```

Метод find()

Метод **find()** требует функцию в качестве параметра, которая может иметь один, два или три параметра для передачи очередного элемента, его индекса и самого массива. Возвращает первый элемент массива, который удовлетворяет условию:

```
function имяФункции(элемент[, индекс[, массив]]) {
  ...
  return ...
}
arr.find(имяФункции);
```

или

```
arr.find(function (элемент[, индекс[, массив]]) {
  ...
  return ...
});
```

Если ни один элемент не соответствует критериям, возвращается значение **undefined**.

Метод find()

Пример

```
let arr = [12, 24, 13, 24, 18, 9];

function search(elem) {
  return elem >= 13 && elem <= 20;
}
alert(arr.find(search)); // результат: 13
```

Метод find()

Пример

```
let arr = [12, 24, 13, 24, 18, 9];

function search(elem) {
  return elem >= 25 && elem <= 30;
}
alert(arr.find(search)); //результат: undefined
```

Метод find()

Пример

```
let arr = [12, 24, 13, 24, 18, 9];

let k = arr.find(function (elem, index) {
    return elem >= 13 && elem <= 20 && index > 2;
});
alert(k); //результат: 18
```

Метод filter()

Метод **filter()** похож на метод **find()**, только возвращает результат в виде массива, в который включает все элементы исходного массива, удовлетворяющие условию, описанному в функции.

Если не найден ни один элемент - возвращается пустой массив.

Метод filter()

Пример

```
let arr = [12, 24, 13, 24, 18, 9];

function search(elem) {
    return elem >= 13 && elem <= 20;
}
alert(arr.filter(search)); // результат: [13, 18]
```

Метод filter()

Пример

```
let arr = [12, 24, 13, 24, 18, 9];

function search1(elem) {
    return elem >= 25 && elem <= 30;
}
alert(arr.filter(search1)); //результат: пустой массив []
```

Метод filter()

Задание 3. Какое значение получит переменная **newArr**?

```
let arr =[12, 17, 12, 67, 23, 23 , 67, 8, 8, 9, 12, 8];

let newArr = arr.filter(function (item, index) {
    return arr.indexOf(item) === index;
});

alert(newArr);
```

Сортировка массива

Для реализации собственного способа сортировки в методе **sort()** нужно создать функцию с двумя параметрами, которая определяет способ сравнения параметров, и передать название функции в качестве параметра **sort()**.

Функции сравнения должна вернуть:

- любое положительное число, чтобы сказать «больше»;
- отрицательное число, чтобы сказать «меньше»;
- 0, чтобы сказать «равно».

Сортировка массива

Метод **sort()** используется для сортировки массива, причем элементы массива при сортировке рассматриваются **как строки**.

Метод реализует лексикографическую сортировку. В результате числа сортируются неправильно, так как все числа при сортировке предварительно преобразуются в строку, а для строк **'110' < '2'**.

```
let arr = [4, 1, 2, 110];  
  
arr.sort();  
  
alert(arr); // 1, 110, 2, 4
```

Сортировка массива

Пример. Создадим функцию **compareNumbers()**, которая сравнивает два элемента как числа. Массив должен сортироваться по возрастанию.

```
// возвращает положительное значение,  
// если число a больше b  
function compareNumbers(a, b) {  
    return Number(a) - Number(b);  
}  
  
let arr = [4, 1, -2, 110];  
  
arr.sort(compareNumbers);  
  
alert(arr); // -2, 1, 4, 110
```

Сортировка массива

Пример. Создадим функцию **compareNumbers()**, которая сравнивает два элемента как числа. Массив должен сортироваться по убыванию.

```
// возвращает положительное значение,  
// если число a больше b  
function compareNumbers(a, b) {  
    return Number(b) - Number(a);  
}  
  
let arr = [4, 1, -2, 110];  
  
arr.sort(compareNumbers);  
  
alert(arr); // -2, 1, 4, 110
```

Обратный порядок массива

Метод **reverse()** переставляет элементы массива в обратном порядке.

Например:

```
let arr = [4, 1, -2, 110 ];  
  
arr.reverse();  
  
alert(arr); // 110, -2, 1, 4
```

Слияние массивов

Для слияния массивов используется метод **concat()**:

- **массив1.concat(массив2)** - присоединяет **массив2** в конец **массива1**;
- **[].concat(массив1, массив2)** - создает новый массив, в котором сначала идет **массив1**, затем **массив2**.

Пример. Создадим новый массив, который является слиянием двух других массивов.

```
let arr = [1, 5, 2, 6, 2];  
  
let arr1 = [4, 3, 12];  
  
let newArr = [].concat(arr, arr1);  
  
// результат: [1, 5, 2, 6, 2, 4, 3, 12]
```

Массивы

Задание 4. Дана строка текста, состоящая из слов и чисел, разделенных пробелом. Вывести в отсортированном виде все различные числа строки.

Многомерные массивы

Массивы в JavaScript могут содержать в качестве элементов другие массивы. Это можно использовать для создания многомерных массивов, например матриц:

```
let matrix = [  
  [1, 2, 3],  
  [4, 5, 6],  
  [7, 8, 9]  
];
```

Многомерные массивы

Обращение к элементам двумерного массива осуществляется с помощью числовых индексов, первый индекс указывает на номер строки, второй – на номер столбца:

```
let k = matrix[0][2]; // k = 3
```

Поскольку двумерный массив представляет собой одномерный массив, в качестве элементов которого используются также одномерные массивы, можно обратиться к строке по индексу:

```
let str = matrix[2]; // str = [7, 8, 9]
```

К строкам двумерного массива можно применять все методы одномерных массивов.

Многомерные массивы

Пример. Выведем матрицу в привычном виде по строкам и столбцам.

```
function alertMatrix(arr) {  
    let str='';  
  
    // перебираем строки матрицы  
    arr.forEach(function (strArr) {  
        str += strArr + "\n";  
    });  
  
    return str;  
}
```

Сравнение массивов

Сравнение массивов осуществляется поэлементно.

Если элементом массива является другой массив - для него тоже применяется поэлементное сравнение.

Следовательно, для сравнения массивов нужно использовать рекурсивную процедуру, которая вызывает саму себя до трех пор, пока ее элементы являются массивами.

Сравнение массивов

Оператор **typeof** позволяет определить, к какому типу относится переменная.

Например:

```
let n = 12.1;
alert(typeof n); // результат: number

let text = 'JavaScript';
alert(typeof text); // результат: string

let isLess = 12 < 16;
alert(typeof isLess); // результат: boolean

let arr = [1, 2, 3];
alert(typeof arr); // результат: object
```

Сравнение массивов

Пример. Создадим рекурсивную процедуру, которая сравнивает два произвольных многомерных массива **a** и **b**.

```
function isEqual(a, b) {
    if (typeof a === "object" && typeof b === "object") {
        if (a.length !== b.length) {
            return false;
        }
        for (let i in a) {
            if (!isEqual(a[i], b[i])) {
                return false;
            }
        }
        return true;
    }
    return a === b;
}
```

Спасибо за внимание!